

2026 찾아가는 학교 컨설팅

4. AI·디지털 문제해결 실무 과정

1부. 문제 정의와 AI 작업환경

강릉오성학교 교사 연수

2026. 9. 30. · 강사 서정완

1부에서 함께 볼 다섯 가지

01 작은 불편에서 시작한다

무엇을 만들지 한 문장으로 정한다

02 AI와 말이 통하는 최소 지식

웹앱·저장·연결·배포를 알아듣는다

03 질문 한 번에서 작업환경으로

AI가 우리 기준을 잊지 않게 한다

04 작업환경을 이루는 것들

규칙·문서·스킬·도구·확인

05 작업공간과 인계

폴더를 만들고 다음 대화로 넘긴다

01

작은 불편에서 시작한다

만들 것은 '앱'이 아니라 '해결할 문제'다

— 01 작은 불편에서 시작한다

검색해서 따라 하던 일을 이제는 도구로 만든다

예: "컴퓨터를 1시간 뒤에 꺼지게 하고 싶다"

예전

- '컴퓨터 예약 종료'를 검색한다
- 명령어를 찾아 복사한다
`shutdown -s -t 3600`
- 시간을 초로 계산하고, 틀리면 다시 검색한다

지금

- AI에게 필요한 것을 말한다
- "30분·1시간·2시간 버튼과 취소 버튼이 있는 프로그램을 만들어 줘"
- 버튼 하나로 예약하고, 버튼 하나로 취소한다

코드를 외우지 않아도, 필요한 것을 분명히 말하면 도구를 만들 수 있다

학교에는 이런 작은 불편이 많다

매일 반복하는 안내

오늘 일정과 준비물을 매번
말로 설명한다

순서가 있는 활동

손 씻기·급식 순서를 매번
그림으로 보여 준다

연습하기 어려운 생활 장면

카페 주문·버스 하차를
실제처럼 연습하기 어렵다

학생마다 다른 자료

같은 활동도 선택지와
그림을 학생별로 바꾼다

수업 중 작은 도구

타이머·차례 정하기·점수판을
찾아 헤맨다

반복되는 문서

비슷한 안내문과 점검표를
매번 새로 만든다

'매번', '다시', '또'가 붙는 일이 좋은 출발점이다

— 01 작은 불편에서 시작한다

좋은 문제는 작고, 분명하고, 확인할 수 있다

- 자주 반복된다
- 누가 쓰는지 분명하다
- 한 문장으로 말할 수 있다
- 작게 시작할 수 있다
- 잘됐는지 확인할 수 있다

피할 예

학생 생활 전반을 관리하는 앱

좋은 예

내릴 정류장에서 하차 벨을 누르는 연습 화면

— 01 작은 불편에서 시작한다

문제를 한 문장으로 적어 본다

누가	어떤 상황에서	무엇을 어려워해서	무엇을 할 수 있게 돕는가
고등부 학생이	카페에서	메뉴를 고르고 주문하는 순서를 어려워해서	3단계 주문을 혼자 반복 연습하게 돕는다
중학부 학생이	버스에서	내릴 곳을 알고 벨을 누르는 것을 어려워해서	안내 방송을 듣고 벨을 누르는 연습을 돕는다

네 칸은 참고 질문이다. 메모로 시작하고, 미정은 만들면서 보완한다

— 01 작은 불편에서 시작한다

무엇을 만들지는 사용 조건이 정한다

조건	생각해 볼 질문	예시 답
누가	학생 혼자? 교사와 함께? 교사만?	학생이 혼자 한다
어디서	교실 PC? 태블릿? 휴대폰?	학교 태블릿, 가로 화면
설치	프로그램을 설치할 수 있나?	설치 불가, 인터넷 주소로 연다
인터넷	항상 연결되어 있나?	교실 와이파이를 쓴다
저장	기록을 남겨야 하나?	남기지 않는다
개인정보	학생 이름·사진이 들어가나?	넣지 않는다

— 01 작은 불편에서 시작한다

조건이 바뀌면 만드는 방식이 바뀐다

쉬움

**학생이 태블릿으로
바로 연다**

설치 없이 주소로 여는 웹
페이지(웹앱)

중간

**학교 PC의
전원을 다룬다**

예약 종료처럼 컴퓨터
기능을 쓰면 설치형
프로그램

어려움

**여러 교사가
기록을 함께 본다**

저장소와 로그인이 필요하고
개인정보 검토가 따라온다

첫 도구는 저장과 로그인이 필요 없는 웹 페이지로 시작한다

02

AI와 말이 통하는 최소 지식

외울 필요는 없다. AI가 묻는 말을 알아들으면 충분하다

보이는 화면, 보이지 않는 처리, 남는 기록

식당에 비유하면 쉽다

프론트엔드 = 홀

사용자가 보고 누르는 화면
(버튼·글자·그림)

백엔드 = 주방

보이지 않는 곳에서
계산하고 요청을 처리하는
부분

데이터베이스 = 창고

기록을 오래 보관하고 여러
기기에서 함께 본다

첫 자료는 화면만 있어도 된다. 기록이 필요하면 그 기기에만 남기는 '브라우저 저장'부터

API와 MCP: 다른 프로그램과 연결하는 약속

API

- 프로그램끼리 정보를 주고받는 창구
- 예: 날씨 앱이 기상 정보를 받아 온다
- 정해진 양식으로 묻고 답을 받는다

MCP

- AI가 외부 도구를 쓰게 하는 표준 연결
- 예: AI가 GitHub·Figma·브라우저를 직접 연다
- 'AI용 USB-C 포트'에 비유된다

연결이 늘수록 AI가 할 수 있는 일도, 조심해야 할 권한도 늘어난다

GitHub와 배포

GitHub

- 파일과 변경 기록을 보관하는 온라인 저장소
- 언제 무엇을 바꿨는지 남는다
- 잘못 바꾸면 이전 버전으로 되돌린다

배포

- 다른 사람이 주소로 접속하게 공개하는 일
- 예: GitHub Pages로 무료 주소 만들기
- 주소를 QR코드로 만들어 태블릿에서 연다

무료 GitHub Pages는 공개 저장소에서 쓰고, 사이트는 누구나 볼 수 있다. 학생 이름·사진은 넣지 않는다

개발 지식은 AI의 질문에 답할 때 쓴다

AI 웹앱과 설치형 중 무엇으로 만들까요?

교사 태블릿에서 바로 열어야 해요.
설치는 못 해요.

AI 기록을 저장해야 하나요?

교사 아니요. 새로고침하면 처음부터
시작해도 돼요.

AI 로그인이 필요한가요?

교사 필요 없어요. 학생 이름도 넣지 않아요.

AI 어디에 공개할까요?

교사 GitHub Pages로 주소만 만들어 주세요.

용어를 정확히 몰라도, 실제 사용할 상황을 알면 답할 수 있다

무엇으로 만들지 모르면 조건을 주고 비교를 부탁한다

아래 조건에 맞는 제작 방법을 2~3가지 비교해 줘.

- 사용: 학생이 학교 태블릿에서 혼자 사용
- 설치: 불가, 인터넷 주소로 열어야 함
- 저장: 필요 없음 / 로그인: 없음

비교 기준: 만드는 난이도, 비용, 고치기 쉬움, 개인정보 위험

마지막에 처음 만드는 교사에게 맞는 방법 하나를 추천해 줘.

기술은 외워서 고르는 것이 아니라, 조건을 주고 판단 근거를 받아 고른다

03

질문 한 번에서 작업환경으로

AI가 우리 기준을 잊지 않게 하는 방법

대화가 길어지면 AI는 기준을 놓친다

머칠에 걸쳐 만들 때 자주 겪는 일

- 어제 설명한 학생 특성을 오늘 다시 설명한다
- '버튼은 크게'라고 했는데 새 화면에서 또 작아진다
- '완료했습니다'라고 했는데 눌러 보면 안 된다
- 한 곳을 고치면 다른 곳이 망가진다

왜 그럴까?

- AI는 전달된 지침·자료·대화 범위에서 작업한다
- 대화가 길어지면 앞 내용이 흐려진다
- 새 대화에 이전 세부사항이 모두 넘어가지는 않는다

좋은 질문 한 번보다 좋은 작업환경이 오래 간다

질문 중심

- 매번 처음부터 다시 설명한다
- 결과가 날마다 달라진다
- 기준이 내 머릿속에만 있다

작업환경 중심

- 규칙과 자료를 파일로 남긴다
- AI가 일하기 전에 파일부터 읽는다
- 기준이 폴더에 남아 누구나 이어 간다

설명은 한 번 파일로 쓰고, 계속 다시 쓴다

하네스: AI가 믿을 만하게 일하도록 갖춘 환경

"사람은 방향을 잡고, 에이전트는 실행한다."

OpenAI 팀이 코드를 직접 쓰지 않고 AI 에이전트로 제품을 만든 뒤 정리한 말

환경을 갖춘다

규칙·자료·도구를 미리
준비한다

의도를 적는다

무엇을 왜 만드는지 분명히
쓴다

확인 구조를 만든다

만들고, 확인하고, 고치는
반복

하네스(harness)는 말의 힘을 원하는 방향으로 쓰게 하는 마구(馬具)에서 온 말이다

결과가 나쁠 때 '더 잘해'라고 하지 않는다

피할 방법

- 버튼이 또 작게 나왔다
- "버튼 크게 해 줘"라고 다시 말한다
- 다음 화면에서 또 작아진다

권하는 방법

- 무엇이 빠져 있었는지 묻는다
- 기준 파일에 "버튼 높이 64px 이상" 한 줄을 적는다
- 이후 모든 화면이 그 기준을 따른다

실패하면 '무엇이 빠져 있었나?'를 묻고, 그 답을 파일에 남긴다

— 03 질문 한 번에서 작업환경으로

해커톤 우승자가 공개한 것은 '작업환경'이었다

공개 사례: Everything Claude Code(ECC)

- 2025년 Anthropic x Forum Ventures 해커톤(뉴욕) 우승자가 만들었다
- Claude Code로 서비스를 만들며 쓰던 설정을 2026년 1월 GitHub에 공개했다
- 규칙·스킬·자동 점검·도구 연결을 한데 묶었다
- 지금은 Codex·Cursor 등 여러 도구용으로도 정리되어 있다

관심

GitHub 별표 26만 개 이상 (2026. 9. 기준)

주의

공식 표준이 아니라 개인이 공개한 사례다. 통째로 설치하기보다 구조를 참고한다

문제를 가장 잘 아는 사람이 도구를 만들었다

Anthropic 'Built with Opus 4.7' 해커톤(2026) 수상자

수상자와 만든 도구

- 의사 출신 개발자 → 진료 연습 도구
- 전자제품 수리 기사 → 수리 진단 도우미
- 컴퓨터과학 교육자 → 계획부터 쓰는 학습 도구
- 코딩 경험 없는 목수의 아들 → 집수리 진단

특별상 · 목수 아버지를 인터뷰해 만든 파일 하나

74% → 81%

숙련 목수의 판단과 일치한 비율

- 진단 규칙·현장 용어·기준 가격·흔한 실수
- 지시문은 그대로 두고 파일만 더했다

교사가 가진 학생 이해도 이런 파일이 될 수 있다

04

작업환경을 이루는 것들

규칙 · 문서 · 스킬 · 에이전트 · 도구 · 확인

작업환경은 학교 업무 체계와 닮았다

구성	학교에 비유하면	실제 모습
규칙	학교 규정·학급 규칙	AGENTS.md
프로젝트 문서	업무 편람·자료철	PROJECT_OVERVIEW.md, DESIGN.md
스킬	업무 매뉴얼	SKILL.md
에이전트	업무 담당자	기획·구현·검토 역할
도구 연결	교무 시스템·장비	MCP (GitHub·Figma 등)
확인과 인계	결재·인수인계서	완료 기준, HANDOFF.md

규칙 파일: 매번 말하지 않아도 지킬 것

카페주문연습/AGENTS.md

- 학생 이름·사진 등 개인정보를 넣지 않는다
- 한 화면에는 행동을 하나만 둔다
- 버튼 높이 64px 이상, 본문 글자 24px 이상
- 요청하지 않은 기능을 더하지 않는다
- 고친 뒤에는 직접 열어 확인하고 결과를 알린다

- 항상 지킬 원칙만 짧게 쓴다
- 길수록 덜 지킨다.
필요 없어진 줄은 지운다
- 같은 부탁을 두 번 했다면
한 줄을 더한다

도구 이름은 달라도 같은 파일을 쓸 수 있다

2026년 9월 기준 공식 문서

구분	Codex	Claude Code	Antigravity
규칙 파일	AGENTS.md	CLAUDE.md (없으면 AGENTS.md)	AGENTS.md 또는 GEMINI.md
스킬 폴더	.agents/skills	.claude/skills	.agents/skills
스킬 부르기	\$이름	/이름	/이름

AGENTS.md와 SKILL.md를 기준으로 두면 도구를 바꿔도 이어 쓸 수 있다

프로젝트 문서: AI가 알아야 할 배경

카페주문연습/PROJECT_OVERVIEW.md

대상: 고등부 학생 (학교 태블릿, 혼자 사용)

목표 행동: 카페에서 음료 1잔 주문하기

단계: 메뉴 고르기 → 크기 고르기 → 주문 확인

이번 버전: 3단계 + 처음부터 다시 하기

하지 않는 것: 결제, 로그인, 기록 저장

- 규칙은 '지켜라', 문서는 '알아둬라'
- '하지 않는 것'을 적으면 AI가 일을 키우지 않는다
- 화면 기준은 DESIGN.md에 따로 둔다 (2부)

스킬: 자주 하는 일을 매뉴얼로 만든다

- 특정 일을 할 때 따라갈 순서를 적은 파일
- 평소에는 이름과 설명만 보고
필요할 때 전체를 읽는다
- 직접 부를 수도, AI가 알아서 고를 수도
있다

```
.agents/skills/screen-check/SKILL.md
```

```
---
```

```
name: screen-check
```

```
description: 학습자료 화면을 점검할 때 쓴다
```

```
---
```

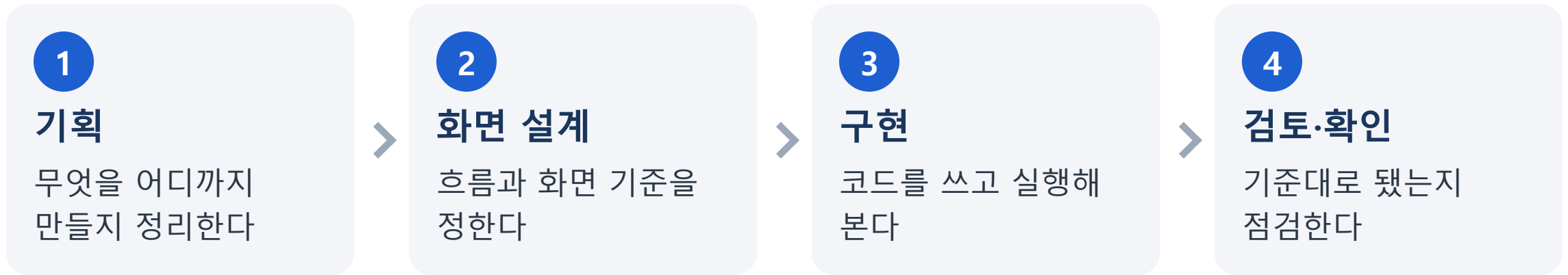
1. 한 화면에 행동이 하나인지 본다
2. 버튼 높이와 글자 크기를 확인한다
3. 잘못 눌렀을 때 돌아갈 수 있는지 본다
4. 문제를 표로 정리해 알린다

규칙은 매번, 스킬은 필요할 때, 혹은 정해진 순간에

구분	언제 쓰이나	누가 부르나	처음에는
규칙 파일	매번, 일을 시작할 때	자동으로 읽는다	꼭 만든다
스킬	그 일이 필요할 때	AI가 고르거나 사람이 이름으로 부른다	같은 일이 반복되면
커맨드·워크플로 (예전 방식)	부를 때만	사람이 /이름으로	대신 스킬로 만든다
혹	정해진 순간 (예: 파일을 고친 직후)	자동으로 실행된다 (AI가 고르지 않는다)	아직 필요 없다

'워크플로'는 일의 순서이고, 그 순서를 담아 두는 곳이 스킬이다

에이전트: 일이 커지면 역할을 나눠 맡긴다



문제가 있으면 앞 단계로 되돌려 다시 한다

작은 자료는 AI 하나로 충분하다. 검토만 새 대화에 맡겨도 효과가 크다

— 04 작업환경을 이루는 것들

도구 연결: AI가 직접 열어 보고 확인하게 한다

ChatGPT의 '앱', Claude의 '커넥터'도 같은 역할을 한다

파일·폴더

프로젝트 파일을 직접 읽고 고친다

GitHub

저장하고, 기록을 남기고, 공개한다

브라우저

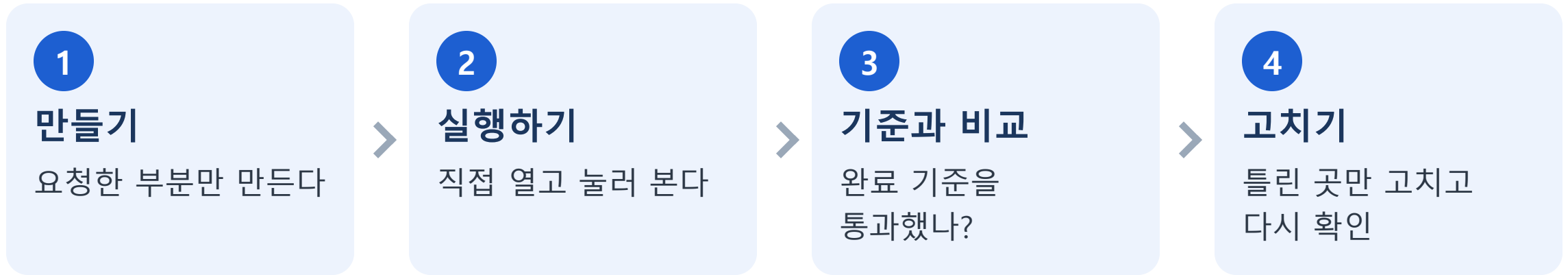
만든 화면을 열어 눌러 보고 확인한다

Figma

디자인을 읽고, 지원 환경에서는 직접 그린다

연결할수록 권한도 커진다. 필요한 것만 연결하고, 학교 계정은 신중히 다룬다

완료 기준을 먼저 주면 AI가 스스로 확인하고 고친다



같은 실패가 세 번 반복되면 멈추고 사람에게 묻는다

'완료했습니다' 대신 '무엇으로 확인했는지'를 받는다

처음에는 이만큼만 갖추면 된다

카페주문연습/	프로젝트 폴더 하나
AGENTS.md	항상 지킬 규칙
PROJECT_OVERVIEW.md	무엇을·왜·어디까지
DESIGN.md	화면 기준 (2부)
HANDOFF.md	지금 상태와 다음 할 일
.agents/skills/	자주 하는 일 (필요할 때)

- ECC처럼 수백 개를 설치하지 않는다
- 같은 문제가 두 번 생기면 그때 한 줄을 더한다

05

작업공간과 인계

폴더 하나로 시작하고, 긴 작업은 파일로 넘긴다

프로젝트마다 폴더 하나를 만든다

내 프로젝트 폴더

문서/AI프로젝트/

찾기 쉬운 곳

카페주문연습/

자료 하나

버스타기연습/

자료 하나

도구 설정 폴더

.codex/

Codex 설정

.claude/

Claude Code 설정

.gemini/

Gemini-Antigravity 설정

AI 도구에서는 '폴더 열기'로 프로젝트 폴더 자체를 연다. 설정 폴더는 직접 만질 일이 적다

공통 규칙과 이 프로젝트의 규칙을 나눈다

모든 프로젝트 공통

- "한국어로 답해 줘"
- "바꾸기 전에 계획부터 보여 줘"

예: ~/.codex/AGENTS.md

이 프로젝트만

- 대상 학생과 사용 기기
- 화면 기준과 하지 않을 것

예: 카페주문연습/AGENTS.md

학교 규칙 → 학급 규칙처럼, 작업 폴더에 가까운 규칙이 더 구체적이다

대화가 길어지면 인계 파일을 쓰고 새 대화로 넘긴다

카페주문연습/HANDOFF.md

지금 상태: 1·2단계(메뉴, 크기) 완성

정한 것: 선택지 2개로 시작, 반응을 보고 늘린다

안 된 방법: 효과음 자동 재생 → 태블릿에서 막힘

다음 할 일: 3단계 주문 확인, '처음부터' 버튼

- 1 기능 하나를 끝낸다
- 2 AI에게 정리를 맡긴다
- 3 새 대화에서 먼저 읽게 한다

'안 된 방법'을 적어야 같은 실패를 되풀이하지 않는다

이 강의자료도 같은 방식으로 만들었다

GitHub의 OSUNGSCHOOL 저장소

AGENTS.md	작업 순서와 우선순위
DESIGN.md	슬라이드 화면 기준
CONTENT_STYLE_GUIDE.md	문구·용어 기준
SPEAKER_SCRIPT_GUIDE.md	발표 대본 기준
docs/	개요·대본·출처·검수 기록
slides/	슬라이드 원본과 PDF

같은 문제가 반복되면 결과물만 고치지 않고 지침 파일을 고친다

처음 요청할 때 쓰는 문장

나는 교사이고, 아래 아이디어를 실제로 만들어 보고 싶어.

(내 메모)

진행에 꼭 필요한 정보가 빠졌다면 중요한 것부터 짧게 질문해 줘.

정하지 않은 내용은 제안하거나 미정으로 남겨 줘.

PROJECT_OVERVIEW.md와 AGENTS.md 초안부터 짧게 정리해 줘.

메모로 시작하고, 꼭 필요한 질문과 짧은 문서 초안부터 정리한다

이어서 작업할 때 쓰는 문장

AGENTS.md와 HANDOFF.md를 먼저 읽고, '다음 할 일' 중 첫 번째만 진행해 줘.

완료 기준: (예: 3단계에서 '처음부터'를 누르면 1단계로 돌아간다)

직접 실행해서 확인하고, 확인한 방법과 결과를 알려 줘.

마지막에 HANDOFF.md를 갱신해 줘.

한 번에 하나, 완료 기준, 확인 결과, 인계 갱신

— 1부 정리

1부에서 가져갈 세 가지

- 1 만들 것은 앱이 아니라 해결할 문제다**
문제와 사용 상황을 메모하고, 필요한 정보를 보완한다
- 2 설명은 대화가 아니라 파일에 남긴다**
규칙·문서·스킬·인계 파일이 AI의 기억이 된다
- 3 완료는 확인한 결과로 판단한다**
완료 기준을 먼저 주고, 확인한 방법을 받는다

2부: 학생이 헤매지 않는 화면을 설계하고 직접 만들어 본다

— 1부 참고 자료

참고 자료 (2026년 9월 확인)

OpenAI, Harness engineering: leveraging Codex in an agent-first world (2026. 2.)

openai.com/index/harness-engineering

OpenAI Developers, Rethinking skills and prompts for GPT-6 Astra (2026. 9. 11.)

developers.openai.com/blog/rethinking-skills-and-prompts-for-gpt-6-astra

OpenAI Codex Docs, AGENTS.md · Agent Skills · Custom prompts

developers.openai.com/codex/guides/agents-md · [/codex/skills](#) · [/codex/custom-prompts](#)

Claude Code Docs, Memory(CLAUDE.md·AGENTS.md) · Skills · Hooks

code.claude.com/docs/en/memory · [/skills](#) · [/hooks-guide](#)

Google Antigravity Docs, Rules · Skills

antigravity.google/docs/rules · antigravity.google/docs/skills

Model Context Protocol, Introduction

modelcontextprotocol.io/docs/getting-started/intro

Claude 블로그, Meet the winners of the Built with Opus 4.7 Claude Code hackathon (2026. 6.)

claude.com/blog/meet-the-winners-of-built-with-opus-4-7-claude-code-hackathon

Everything Claude Code(ECC) 저장소 · GitHub Docs, About GitHub Pages

github.com/affaan-m/ECC · docs.github.com/pages